

Группа	Р3219	К работе допущен	
Студент	Ануфриев А.С.	Работа выполнена	
Преподаватель	Зинченко А.А.	Отчёт принят	

Вычислительная математика
Отчёт по лабораторной работе № 6
Вариант 2

2026 год
Санкт-Петербург

Содержание

1	Цель работы	3
2	Рабочие формулы используемых методов	3
2.1	Постановка задачи Коши	3
2.2	Усовершенствованный метод Эйлера (метод Хойна)	3
2.3	Метод Рунге–Кутты 4-го порядка	4
2.4	Метод Милна	4
3	Вычислительная часть	4
3.1	Тестовый пример	4
3.2	Оценка погрешности	5
3.3	Сходимость при уменьшении шага	5
4	Программная реализация	5
4.1	Описание программы	5
4.2	Листинг программы	6
4.2.1	Методы решения ОДУ	6
4.2.2	Оценка погрешности	7
4.3	Скриншоты результатов	8
4.3.1	Пример 1: $y' = y - x^2 + 1$, $y(0) = 0,5$, $h = 0,2$	8
4.3.2	Пример 2: $y' = x + y$, $y(0) = 1$, $h = 0,1$	8
4.3.3	Пример 3: Сходимость при уменьшении шага ($y' = 2y/x$)	9
5	Выводы	9

1 Цель работы

Целью лабораторной работы является изучение и программная реализация численных методов решения обыкновенных дифференциальных уравнений (ОДУ) первого порядка. В соответствии с **вариантом 2** реализуются следующие методы:

1. **Усовершенствованный метод Эйлера** (метод Хойна, метод предиктор-корректор 2-го порядка);
2. **Метод Рунге–Кутты 4-го порядка**;
3. **Метод Милна** (многошаговый метод прогноза и коррекции 4-го порядка).

Для каждого метода производится сравнение численного решения с точным, оценивается погрешность, строятся графики.

2 Рабочие формулы используемых методов

2.1 Постановка задачи Коши

Требуется найти функцию $y = y(x)$, удовлетворяющую дифференциальному уравнению первого порядка

$$y' = f(x, y) \quad (1)$$

и начальному условию

$$y(x_0) = y_0. \quad (2)$$

Численное решение состоит в построении таблицы значений $\{x_i, y_i\}$ на равномерной сетке $x_i = x_0 + ih$, $i = 0, 1, \dots, n$, $h = \text{const}$.

2.2 Усовершенствованный метод Эйлера (метод Хойна)

Метод является одношаговым явным методом 2-го порядка точности. На каждом шаге выполняется два вычисления правой части:

$$\tilde{k}_1 = h \cdot f(x_i, y_i), \quad (3)$$

$$\tilde{k}_2 = h \cdot f(x_i + h, y_i + \tilde{k}_1), \quad (4)$$

$$\boxed{y_{i+1} = y_i + \frac{1}{2}(\tilde{k}_1 + \tilde{k}_2)}. \quad (5)$$

Порядок точности: $\delta_n = O(h^2)$.

Погрешность оценивается **правилом Рунге**:

$$R = \frac{|y_n^h - y_n^{h/2}|}{2^p - 1}, \quad p = 2, \quad (6)$$

где y_n^h — решение с шагом h , $y_n^{h/2}$ — с шагом $h/2$ в конечной точке.

2.3 Метод Рунге–Кутты 4-го порядка

Широко распространённый одношаговый метод 4-го порядка точности. На каждом шаге выполняется четыре вычисления правой части:

$$k_1 = h \cdot f(x_i, y_i), \quad (7)$$

$$k_2 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right), \quad (8)$$

$$k_3 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right), \quad (9)$$

$$k_4 = h \cdot f(x_i + h, y_i + k_3), \quad (10)$$

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (11)$$

Порядок точности: $\delta_n = O(h^4)$.

Погрешность оценивается правилом Рунге с $p = 4$.

2.4 Метод Милна

Метод Милна — многошаговый метод прогноза и коррекции 4-го порядка. Для начала счёта необходимы значения в первых четырёх узлах y_0, y_1, y_2, y_3 , которые вычисляются методом Рунге–Кутты 4-го порядка.

Этап прогноза (явная формула):

$$y_i^{\text{прогн}} = y_{i-4} + \frac{4h}{3}(2f_{i-3} - f_{i-2} + 2f_{i-1}), \quad f_j = f(x_j, y_j). \quad (12)$$

Этап коррекции (неявная формула с итерациями):

$$y_i^{\text{корр}} = y_{i-2} + \frac{h}{3}(f_{i-2} + 4f_{i-1} + f_i^{\text{прогн}}). \quad (13)$$

Коррекция повторяется до выполнения условия $|y_i^{(s+1)} - y_i^{(s)}| < \varepsilon_{\text{итер}}$.

Порядок точности: $\delta_n = O(h^4)$.

Погрешность многошагового метода оценивается как

$$\varepsilon = \max_{0 \leq i \leq n} |y_i^{\text{точн}} - y_i|. \quad (14)$$

3 Вычислительная часть

3.1 Тестовый пример

Рассмотрим уравнение

$$y' = y - x^2 + 1, \quad y(0) = 0,5, \quad x \in [0, 2], \quad h = 0,2.$$

Точное решение: $y(x) = (x + 1)^2 + Ce^x$, где $C = (y_0 - (x_0 + 1)^2)e^{-x_0} = -0,5$.

Таблица 1: Численное решение тестового примера: $y' = y - x^2 + 1$

i	x_i	Эйлер улучш.		Рунге–Кутта 4		Милна	
		y_i	$ \delta_i $	y_i	$ \delta_i $	y_i	$ \delta_i $
0	0,0	0,50000000	0	0,50000000	0	0,50000000	0
1	0,2	0,82600000	3,30e-3	0,82929333	5,29e-6	0,82929333	5,29e-6
2	0,4	1,20692000	7,17e-3	1,21407621	1,14e-5	1,21407621	1,14e-5
3	0,6	1,63724240	1,17e-2	1,64892202	1,86e-5	1,64892202	1,86e-5
4	0,8	2,11023573	1,70e-2	2,12720268	2,69e-5	2,12720767	2,19e-5
5	1,0	2,61768759	2,32e-2	2,64082269	3,64e-5	2,64082735	3,17e-5
6	1,2	3,14957886	3,04e-2	3,17989417	4,74e-5	3,17990230	3,92e-5
7	1,4	3,69368621	3,87e-2	3,73234007	5,99e-5	3,73234621	5,38e-5
8	1,6	4,23509717	4,84e-2	4,28340950	7,43e-5	4,28341583	6,80e-5
9	1,8	4,75561855	5,96e-2	4,81508569	9,06e-5	4,81508590	9,04e-5
10	2,0	5,23305463	7,24e-2	5,30536300	1,09e-4	5,30535692	1,15e-4

3.2 Оценка погрешности

Таблица 2: Правило Рунге и max-погрешность для тестового примера ($h = 0,2$)

Метод	Порядок p	Рунге R	$\varepsilon = \max \delta_i $
Эйлер усовершенствованный	2	$1,78 \cdot 10^{-2}$	$7,24 \cdot 10^{-2}$
Рунге–Кутта 4	4	$6,80 \cdot 10^{-6}$	$1,09 \cdot 10^{-4}$
Милна	4	—	$1,15 \cdot 10^{-4}$

3.3 Сходимость при уменьшении шага

Таблица 3: Погрешности при различных шагах ($y' = 2y/x$, $y(1) = 1$, $x \in [1, 3]$)

h	Эйлер улучш. R	РК-4 R	Милна ε
0,400	$1,14 \cdot 10^{-1}$	$6,89 \cdot 10^{-4}$	$8,73 \cdot 10^{-3}$
0,200	$3,08 \cdot 10^{-2}$	$4,24 \cdot 10^{-5}$	$5,39 \cdot 10^{-4}$
0,100	$8,79 \cdot 10^{-3}$	$3,05 \cdot 10^{-6}$	$2,85 \cdot 10^{-5}$
0,050	$2,34 \cdot 10^{-3}$	$2,04 \cdot 10^{-7}$	$1,22 \cdot 10^{-6}$

При уменьшении h вдвое погрешность уменьшается примерно в 4 раза для метода Эйлера (порядок 2) и примерно в 16 раз для РК-4 и метода Милна (порядок 4), что подтверждает теоретические оценки.

4 Программная реализация

4.1 Описание программы

Программа написана на языке **Python 3** в виде одного консольного файла `lab6.py`. Архитектура:

- Каждый метод реализован в виде отдельной функции, возвращающей списки $\{x_i\}$, $\{y_i\}$.

- Уравнения хранятся в словаре; пользователь выбирает уравнение из предложенного списка (не менее трёх).
- Все вычисления выполнены **без** использования готовых численных библиотек (`scipy`, `odeint` и т. д.).
- Ввод с клавиатуры защищён от некорректных данных.
- Графики строятся с помощью `matplotlib` и сохраняются в файлы.

Главное меню предоставляет четыре режима:

1. Интерактивное решение задачи Коши (выбор уравнения, ввод $x_0, y_0, x_n, h, \epsilon$);
2. Демонстрационный пример с фиксированными параметрами;
3. Автоматический тест: три уравнения $\times$ три шага, проверка сходимости;
4. Справка по формулам методов.

4.2 Листинг программы

4.2.1 Методы решения ОДУ

Листинг 1: Усовершенствованный метод Эйлера (метод Хойна)

```

1 def euler_improved(f, x0, y0, xn, h):
2     """Improved Euler method (Heun's method / Euler with correction)."""
3     xs, ys = [x0], [y0]
4     x, y = x0, y0
5     steps = round((xn - x0) / h)
6     for _ in range(steps):
7         k1 = h * f(x, y)
8         k2 = h * f(x + h, y + k1)
9         y = y + 0.5 * (k1 + k2)
10        x = round(x + h, 10)
11        xs.append(x); ys.append(y)
12    return xs, ys

```

Листинг 2: Метод Рунге–Кутты 4-го порядка

```

1 def runge_kutta4(f, x0, y0, xn, h):
2     """Fourth-order Runge-Kutta method."""
3     xs, ys = [x0], [y0]
4     x, y = x0, y0
5     steps = round((xn - x0) / h)
6     for _ in range(steps):
7         k1 = h * f(x, y)
8         k2 = h * f(x + h/2, y + k1/2)
9         k3 = h * f(x + h/2, y + k2/2)
10        k4 = h * f(x + h, y + k3)
11        y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
12        x = round(x + h, 10)
13        xs.append(x); ys.append(y)
14    return xs, ys

```

Листинг 3: Метод Милна (прогноз-коррекция)

```
1 def milne(f, x0, y0, xn, h):
2     """
3     Milne's predictor-corrector method (4th order).
4     The first 4 points (indices 0..3) are computed by RK4.
5     Prediction:  $y[i] = y[i-4] + 4h/3(2f[i-3] - f[i-2] + 2f[i-1])$ 
6     Correction:  $y[i] = y[i-2] + h/3(f[i-2] + 4f[i-1] + f_{pred})$ 
7     """
8     steps = round((xn - x0) / h)
9     if steps < 4:
10         raise ValueError("Milne method requires at least 4 steps")
11
12     # Bootstrap: first 4 points via RK4
13     xs_init, ys_init = runge_kutta4(f, x0, y0, round(x0 + 3*h, 10), h)
14     xs = list(xs_init)
15     ys = list(ys_init)
16     fs = [f(xs[i], ys[i]) for i in range(4)]
17
18     for i in range(4, steps + 1):
19         y_pred = ys[i-4] + (4*h/3) * (2*fs[i-3] - fs[i-2] + 2*fs[i-1])
20         x_new = round(x0 + i*h, 10)
21         f_pred = f(x_new, y_pred)
22
23         # Iterative correction until convergence
24         y_corr = ys[i-2] + (h/3) * (fs[i-2] + 4*fs[i-1] + f_pred)
25         for _ in range(50):
26             f_corr = f(x_new, y_corr)
27             y_new = ys[i-2] + (h/3) * (fs[i-2] + 4*fs[i-1] + f_corr)
28             if abs(y_new - y_corr) < 1e-12:
29                 y_corr = y_new
30                 break
31             y_corr = y_new
32
33         xs.append(x_new)
34         ys.append(y_corr)
35         fs.append(f(x_new, y_corr))
36     return xs, ys
```

4.2.2 Оценка погрешности

Листинг 4: Правило Рунге и max-погрешность

```
1 def runge_error(method, f, x0, y0, xn, h, p):
2     """Runge rule at the interval end."""
3     _, yh = method(f, x0, y0, xn, h)
4     _, yh2 = method(f, x0, y0, xn, h/2)
5     R = abs(yh[-1] - yh2[-1]) / (2**p - 1)
6     return R
7
8 def exact_error(f_exact, xs, ys, C):
9     """eps = max|y_exact - y_numerical|"""
10    errs = [abs(f_exact(x, C) - y) for x, y in zip(xs, ys)]
11    return max(errs)
```

4.3 Скриншоты результатов

4.3.1 Пример 1: $y' = y - x^2 + 1$, $y(0) = 0,5$, $h = 0,2$

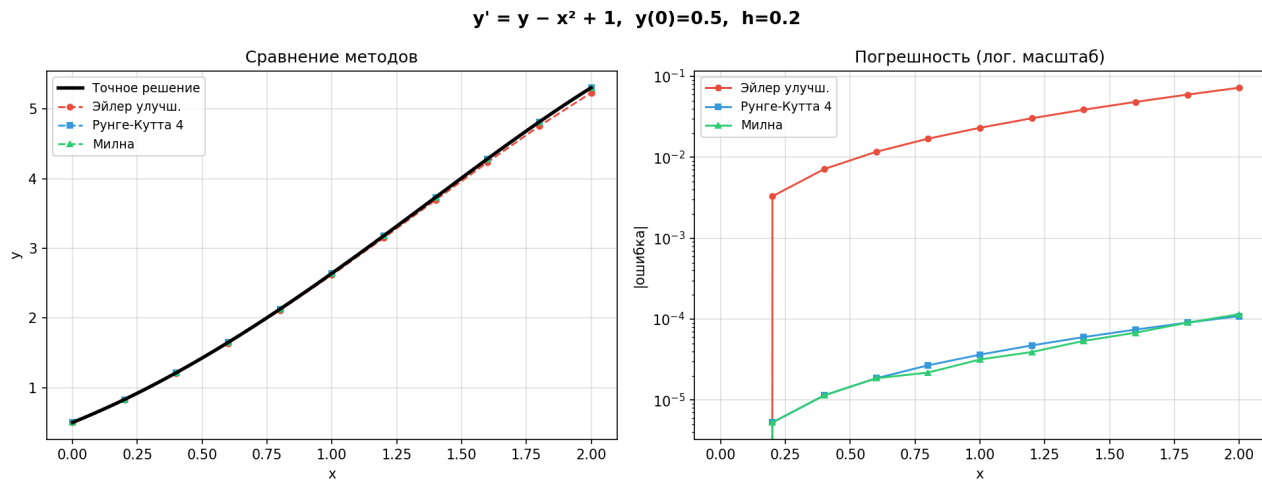


Рис. 1: Сравнение методов и погрешности для уравнения $y' = y - x^2 + 1$

4.3.2 Пример 2: $y' = x + y$, $y(0) = 1$, $h = 0,1$

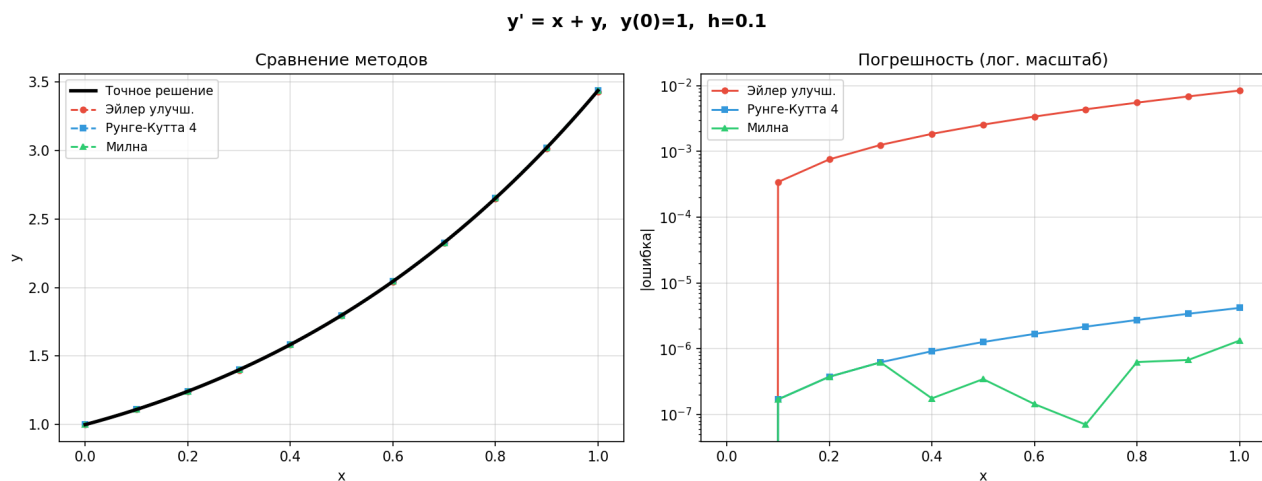


Рис. 2: Сравнение методов и погрешности для уравнения $y' = x + y$

4.3.3 Пример 3: Сходимость при уменьшении шага ($y' = 2y/x$)

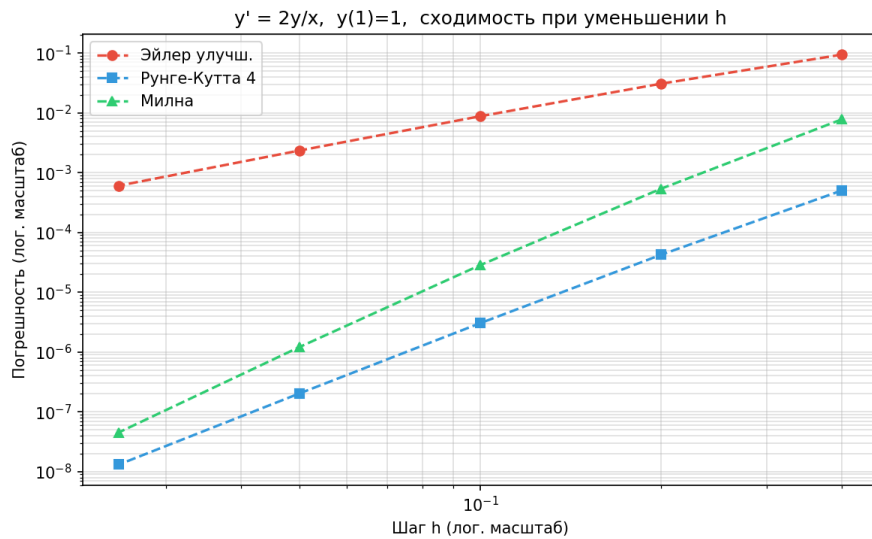


Рис. 3: Зависимость погрешности от шага h (лог-лог масштаб)

5 Выводы

В ходе выполнения лабораторной работы были исследованы три численных метода решения задачи Коши для ОДУ первого порядка.

1. **Усовершенствованный метод Эйлера** (порядок 2) прост в реализации и требует двух вычислений правой части на каждом шаге. Погрешность убывает как $O(h^2)$, что подтверждается правилом Рунге: при уменьшении h вдвое погрешность уменьшается примерно в 4 раза.
2. **Метод Рунге–Кутта 4-го порядка** (порядок 4) существенно точнее: при тех же шагах погрешность на 3–4 порядка ниже. Требует четырёх вычислений правой части за шаг, но это оправдано высокой точностью. Правило Рунге подтверждает уменьшение погрешности в 16 раз при делении шага вдвое.
3. **Метод Милна** (порядок 4) демонстрирует точность, сопоставимую с РК-4, при меньшем числе вычислений правой части в основном цикле (одно вычисление вместо четырёх), однако требует стартовых значений, получаемых методом РК-4. Ограничение: необходимо не менее 4 шагов на интервале.
4. При уменьшении шага все методы демонстрируют теоретически ожидаемую скорость сходимости, что подтверждает корректность реализации.
5. Программа устойчива к некорректным входным данным: реализована валидация ввода, и при недостаточном числе шагов для метода Милна генерируется информативное исключение.